Fluxo de criacao de pedido

Chatbot WhatsApp — diagramas + referencia de codigo

renthus-chat-erp — gerado 2026-04-17

As figuras seguem o estilo arquitetura (fundo escuro, caixas, setas). Texto ASCII nos rotulos por limitacao de fonte no PDF; o significado esta nos ficheiros indicados.

Figura 1 — Ingresso e worker

Replica a estrutura do teu print: WhatsApp -> fila -> cron processa -> processInboundMessage.

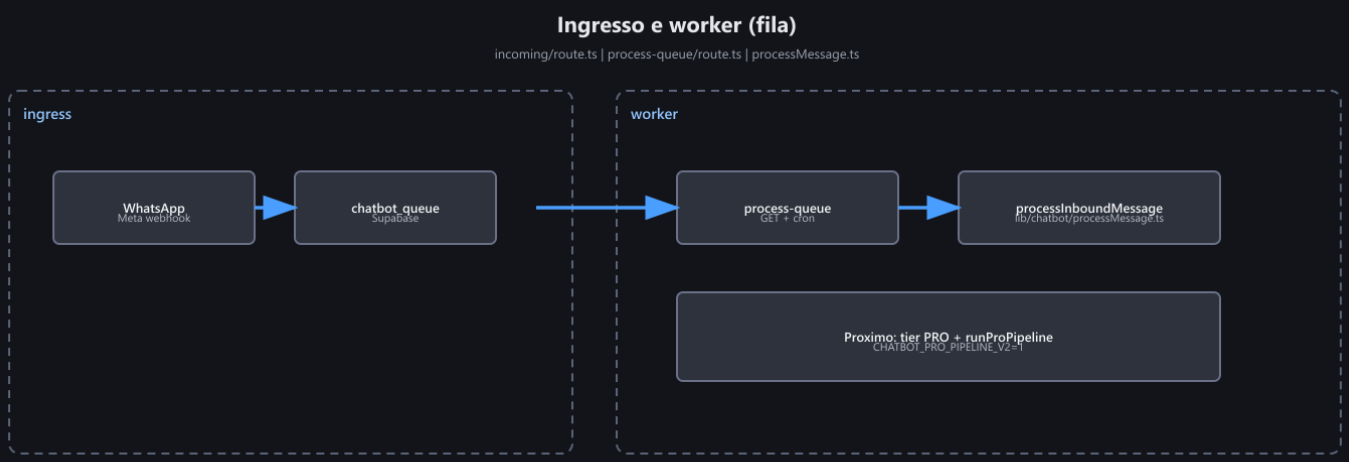


Figura 2 — Ramo PRO vs legado

processMessage.ts: se plano PRO e CHATBOT_PRO_PIPELINE_V2=1 corre deps.factory + runProPipeline; senao inboundPipeline.

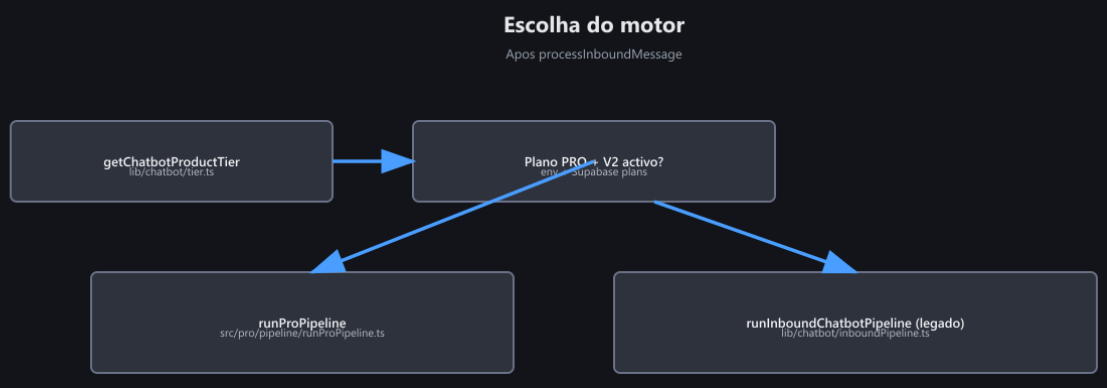


Figura 3 — Stages do runProPipeline

Ordem em runProPipeline.ts. persistAndEmit em TODOS os exits (curtos ou fluxo completo).

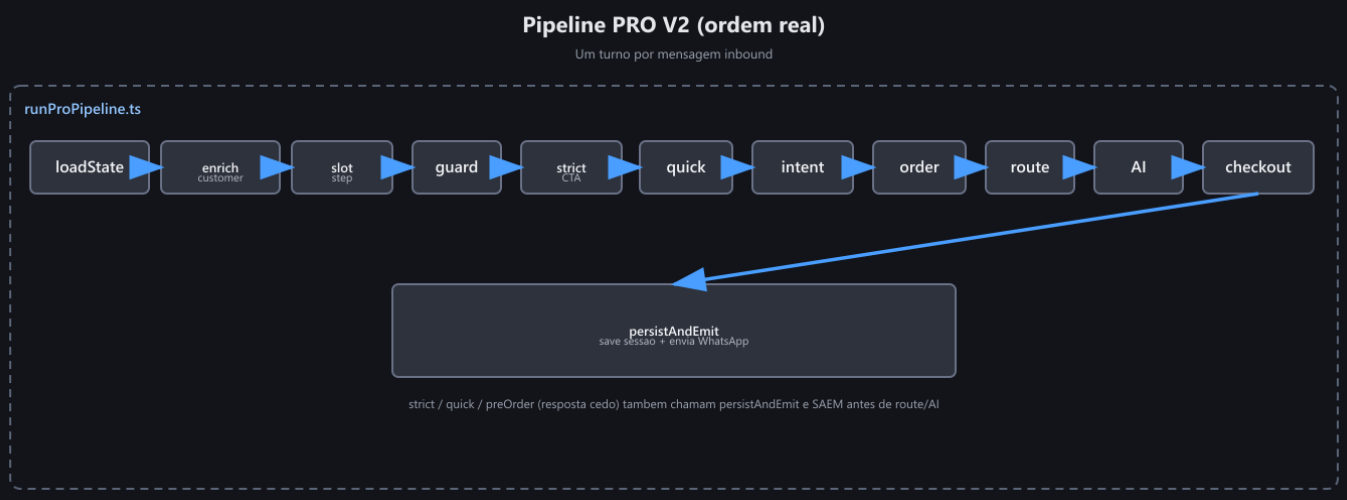
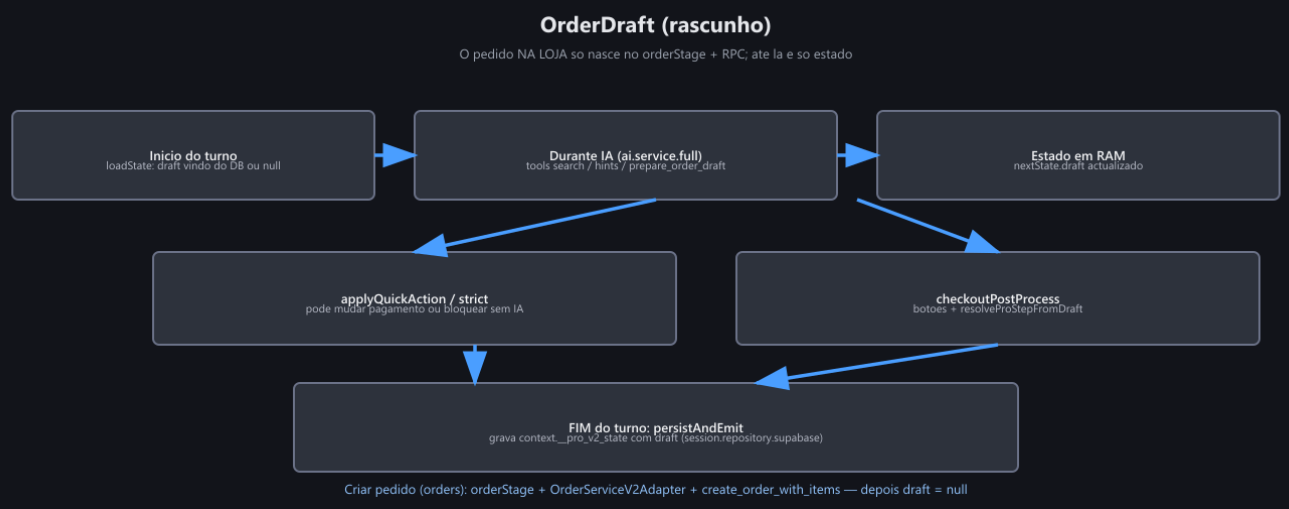


Figura 3b — Ciclo do OrderDraft

Rascunho = OrderDraft (contracts.ts). Pedido real = RPC so no orderStage.



Detalhe: o que cada caixa faz (1/2)

loadState (stages/loadState.ts): le chatbot_sessions; devolve ProSessionState incl. draft (ou null se nunca houve PRO neste thread).

enrich (enrichCustomerFromPhone.ts): preenche customerId pelo telefone; nao constroi catalogo nem draft.

slot (orderSlotStep.ts): withResolvedSlotStepUnlessAwaitingConfirmation — alinha ProStep ao draft (ex.: itens+endereco sem pagamento => awaiting address). Nao altera items do draft.

guard (stages/guardRails.ts): bloqueia inbound vazio ou conversa em handover. Sem alteracao de draft.

strict (checkoutPostProcess.ts strictCheckoutStructuredGate): se pagamento/endereco exigem botoes, responde sem IA; draft normalmente igual.

quick (applyQuickAction): ids pro_pay_*, confirmar endereco, cancelar, etc. AQUI altera-se draft (pagamento) ou limpa-se tudo (cancelar).

intent (intentStage + intentClassifier): so classifica mensagem (order_intent, human, ...). NAO le nem escreve draft.

Detalhe: o que cada caixa faz (2/2)

order (orderStage.ts): so com step=pro_awaiting_confirmation + texto/botao de confirmacao explicita. AQUI corre createFromDraft (order.service.v2) => RPC create_order_with_items. Sucesso: step idle + draft=null. Falha: mensagem + step pode voltar a collecting.

route (routeStage.ts): human_intent => handover; catalogo/status => flow ou texto; senao mode ai ou direct. Pode mudar step em handover.

AI (aiStage + ai.service.full.ts): Claude + tools. prepare_order_draft (prepareOrderDraft.ts) valida itens/endereco/pagamento no servidor e devolve OrderDraft canónico. Draft fica no estado em memoria ate persistAndEmit.

checkout (checkoutPostProcess): acrescenta botoes WhatsApp e resolveProStepFromDraft no estado.

persistAndEmit (stages/persistAndEmit.ts): sessionRepo.save grava __pro_v2_state; messageGateway envia cada OutboundMessage.

Figura 4 — ProStep ate fecho

orderSlotStep.ts alinha step ao draft; orderStage chama RPC quando cliente confirma.

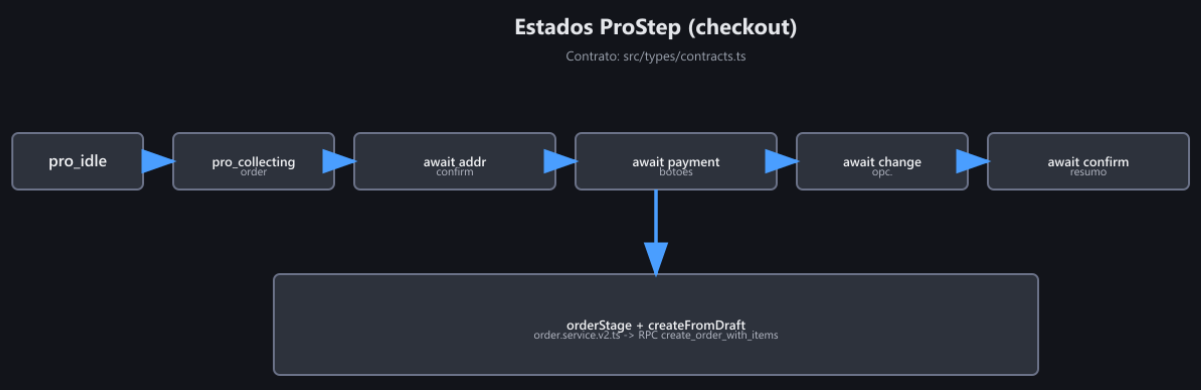
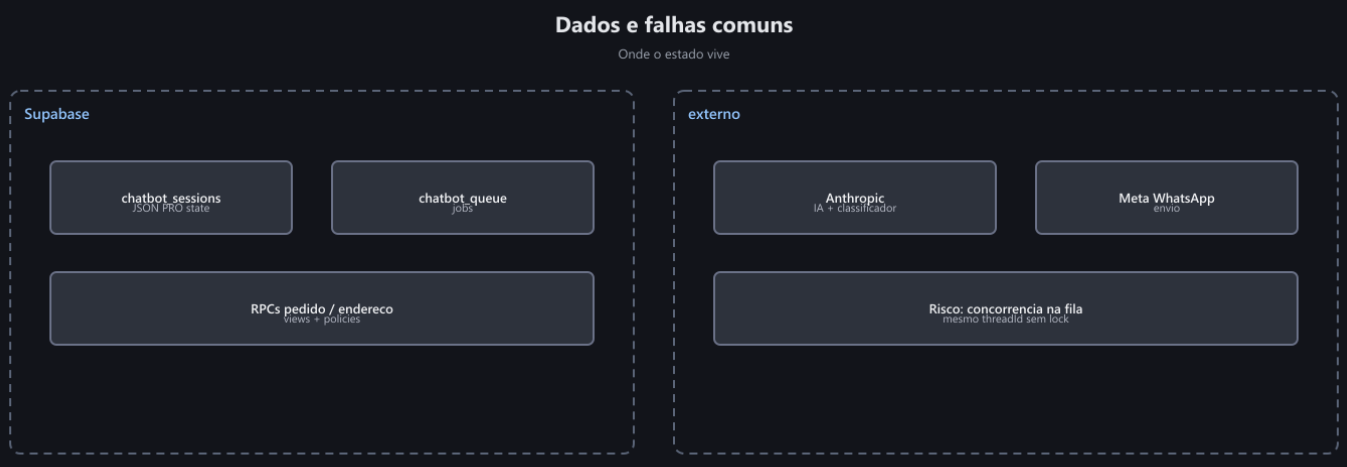


Figura 5 — Pedido na base

order.service.v2.ts: validacao, endereco, create_order_with_items.

Figura 6 — Dados e escala

Persistencia de sessao; fila; risco de corrida sem serializacao por thread.



Ficheiros-chave (indice)

Webhook + fila: app/api/whatsapp/incoming/route.ts | app/api/chatbot/process-queue/route.ts

Entrada motor: lib/chatbot/processMessage.ts | lib/chatbot/tier.ts

Pipeline PRO: src/pro/pipeline/runProPipeline.ts | deps.factory.ts | stages/*.ts

Rascunho + tools: lib/chatbot/pro/prepareOrderDraft.ts | searchProdutos.ts | orderHints.ts

IA: src/pro/adapters/ai/ai.service.full.ts | stripModelIntentSuffix.ts

Checkout CTAs: src/pro/pipeline/stages/checkoutPostProcess.ts

Contratos: src/types/contracts.ts

Regenerar este PDF: node scripts/generateFluxoPedidoChatbotPdf.mjs